

Information security – tutorial 6

1 – Digital Signatures

- Key Generation

RSA keys were created using OpenSSL with a size of 3072 bits. A bigger key size means better security because it is harder for attackers to break it. The public exponent 65537 is used because it is fast and secure.

```
hiba@hiba-VirtualBox: ~/Downloads$ openssl genrsa 3072 > hiba-sig.keys
hiba@hiba-VirtualBox: ~/Downloads$ openssl rsa -in hiba-sig.keys -text -noout
Private-Key: (3072 bit, 2 primes)
modulus:
    00:eb:99:a5:ef:09:23:65:d9:99:1b:c0:e0:a7:7b:
    26:81:e2:08:e0:3d:09:84:2c:cb:f6:41:b3:3b:14:
    fa:9d:e4:e0:15:21:33:5a:df:d2:2b:1b:1f:0e:73:
    b8:9e:96:0f:7b:72:aa:5c:0f:51:9b:b9:1f:e9:dd:
    0a:41:26:a9:b5:15:00:03:dd:e1:be:73:90:e0:96:
    bf:b9:52:f8:0d:37:4c:77:0e:00:bc:2f:b4:b7:3b:
    0b:ee:fa:08:da:56:be:7b:df:fa:0b:05:1a:f9:11:
    7c:ad:39:29:0d:a5:e2:a6:0b:a7:cb:01:5e:fs:07:
    95:ca:a7:3d:72:41:aa:bc:04:b7:c4:47:a8:ba:0f:
    97:3a:0d:bd:a4:45:1e:22:5c:ec:fe:73:0b:ce:bd:
    bf:1a:e1:59:04:27:12:0e:5a:51:f3:05:38:31:0b:
    65:15:df:55:23:71:fd:f8:a5:03:bd:2b:3a:03:b4:
    f4:b2:fe:77:79:40:a2:a0:a3:7d:7e:02:04:96:04:
    0c:08:03:02:bcc4c:fe:c0:31:71:00:7f:04:0e:
    76:fe:fa:04:08:1cc:c5:21:00:f9:10:4f:c2:04:bd:
    3b:79:78:23:32:3f:79:35:f4:0b:ce:0b:0b:0d:cd:
    d9:4b:ce:ce:0e:93:46:38:4c:c7:dd:ef:84:57:a8:
    2c:61:d1:0b:05:b7:02:38:1e:a0:95:75:d0:07:e5:
    17:e3:ef:b0:09:ef:fc:f9:5b:f1:09:c0:51:fo:20:
    6d:99:cf:40:d7:71:37:0c:3a:35:9b:3b:3b:bd:77:
    43:f2:01:0d:77:26:e9:11:f9:f6:d6:04:a5:ab:70:
    f4:21:c3:06:f3:18:03:be:b5:93:ef:e1:a7:43:99:
    37:ee:a1:fd:7f:c5:7b:7b:03:5f:ee:a3:7a:11:ee:
    a1:c0:42:5b:70:b7:32:9a:0a:c3:7a:d0:22:7c:76:
    e8:f0:cc:ff:0a:38:44:b7:a5:00:e0:d0:d2:ed:47:
    f1:50:cb:0f:0a:b0:04:f9:a1:ed
publicExponent: 65537 (0x10001)
privateExponent:
    28:e3:d1:3b:5e:7e:37:9b:ba:04:1a:d0:01:e9:7b:
    63:9c:73:c3:ba:2f:09:c9:ab:2b:ff:d5:02:26:43:
    9a:59:39:53:d9:cb:36:eb:03:05:0f:4b:e7:7a:16:
    32:ee:77:ee:56:f0:0b:00:74:0a:bd:1d:7c:0b:75:
    7b:96:e7:2f:0a:cf:5d:0b:30:71:a2:ef:74:f4:6d:
    0a:7d:9e:ab:79:0a:ce:b0:0e:20:03:e4:09:a5:5e:
    5c:db:a1:d1:41:3c:04:54:3e:7d:31:be:d2:7c:73:
    7d:47:3b:08:f3:3b:03:cf:a0:bc:0a:0a:ed:5a:
    32:cf:ee:0f:16:45:bc:bd:06:bd:eb:27:07:34:6e:
    da:0c:04:0a:f3:af:32:42:ce:0b:72:c9:eb:79:41:
    9a:12:0d:0a:1a:5e:47:cb:2a:17:0d:70:cb:b7:ff:
    7b:a0:32:df:00:e1:00:a0:c1:ce:5d:c3:7a:92:ff:
    7a:20:58:2a:2e:8b:c6:51:9a:09:39:cc:fd:fd:3b:
    6d:0d:fa:1e:20:6d:c1:0b:c3:7c:2c:74:ee:79:0e:
    07:03:b0:04:0d:25:97:b7:49:9a:a5:e4:4e:e9:
    6a:79:4e:c8:49:b9:41:c5:0a:dc:cd:b0:fa:0f:5c:
    (Exponent: 65537 (0x10001))

Writing the key
hiba@hiba-VirtualBox: ~/Downloads$ openssl rsa -in hiba-sig.keys -text -noout -pubin
Public-Key: (3072 bit)
Modulus:
    00:eb:99:a5:ef:09:23:65:d9:99:1b:c0:e0:a7:7b:
    26:81:e2:08:e0:3d:09:84:2c:cb:f6:41:b3:3b:14:
    fa:9d:e4:e0:15:21:33:5a:df:d2:2b:1b:1f:0e:73:
    b8:9e:96:0f:7b:72:aa:5c:0f:51:9b:b9:1f:e9:dd:
    0a:41:26:a9:b5:15:00:03:dd:e1:be:73:90:e0:96:
    bf:b9:52:f8:0d:37:4c:77:0e:00:bc:2f:b4:b7:3b:
    0b:ee:fa:08:da:56:be:7b:df:fa:0b:05:1a:f9:11:
    7c:ad:39:29:0d:a5:e2:a6:0b:a7:cb:01:5e:fs:07:
    95:ca:a7:3d:72:41:aa:bc:04:b7:c4:47:a8:ba:0f:
    97:3a:0d:bd:a4:45:1e:22:5c:ec:fe:73:0b:ce:bd:
    bf:1a:e1:59:04:27:12:0e:5a:51:f3:05:38:31:0b:
    65:15:df:55:23:71:fd:f8:a5:03:bd:2b:3a:03:b4:
    f4:b2:fe:77:79:40:a2:a0:a3:7d:7e:02:04:96:04:
    0c:08:03:02:bcc4c:fe:c0:31:71:00:7f:04:0e:
    76:fe:fa:04:08:1cc:c5:21:00:f9:10:4f:c2:04:bd:
    3b:79:78:23:32:3f:79:35:f4:0b:ce:0b:0b:0d:cd:
    d9:4b:ce:ce:0e:93:46:38:4c:c7:dd:ef:84:57:a8:
    2c:61:d1:0b:05:b7:02:38:1e:a0:95:75:d0:07:e5:
    17:e3:ef:b0:09:ef:fc:f9:5b:f1:09:c0:51:fo:20:
    6d:99:cf:40:d7:71:37:0c:3a:35:9b:3b:3b:bd:77:
    43:f2:01:0d:77:26:e9:11:f9:f6:d6:04:a5:ab:70:
    f4:21:c3:06:f3:18:03:be:b5:93:ef:e1:a7:43:99:
    37:ee:a1:fd:7f:c5:7b:7b:03:5f:ee:a3:7a:11:ee:
    a1:c0:42:5b:70:b7:32:9a:0a:c3:7a:d0:22:7c:76:
    e8:f0:cc:ff:0a:38:44:b7:a5:00:e0:d0:d2:ed:47:
    f1:50:cb:0f:0a:b0:04:f9:a1:ed
Public-Key: (3072 bit)
Exponent: 65537 (0x10001)
```

- Signing and Verification

A file was signed using the private key and verified using the public key. The verification was successful, which confirms that the file has not been modified and that it comes from a trusted source.

```

hibaghiba-VMware20-1:~/Downloads$ openssl genrsa 3072 > hiba-sig.keys
hibaghiba-VMware20-1:~/Downloads$ openssl pkeyutl -sign -in input -out input.sig -inkey
hiba-sig.keys -digest SHA-256 -rawin
hibaghiba-VMware20-1:~/Downloads$ openssl rsa -in hiba-sig.keys -pubout -out hiba-sig.pu
b
writing RSA key
hibaghiba-VMware20-1:~/Downloads$ openssl pkeyutl -verify -in input -inkey hiba-sig.pub
-pubin -digest SHA-256 -rawin -sigfile input.sig
Signature Verified Successfully

```

Relationship between input size and key size:

The input file can be larger than the RSA key size because OpenSSL signs the hash of the file (SHA-256), not the entire file. The size of the signature file depends on the RSA key size. For a 3072-bit key, the signature is 384 bytes. Therefore, large files can be signed without issues when hashing is used.

- File Modification Test

```

hibaghiba-VMware20-1:~/Downloads$ echo "I changed the file" >> input
hibaghiba-VMware20-1:~/Downloads$ openssl pkeyutl -verify -in input -inkey hiba-sig.pub
-pubin -digest SHA-256 -rawin -sigfile input.sig
Signature Verification Failure
C03CBAAE3DE70000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa
/rsa_sign.c:442:
C03CBAAE3DE70000:error:1C800004:Provider routines:rsa_verify_directly:RSA lib:../provide
rs/implementations/signature/rsa_sig.c:1041:

```

This happens because digital signatures depend on a hash value. If anything changes, the hash changes completely, so the signature becomes invalid.

- Wrong Key Test

When using a different public key, the verification failed. This shows that only the correct key pair can validate the signature.

```

hibaghiba-VMware20-1:~/Downloads$ openssl genrsa 3072 > fake.keys
hibaghiba-VMware20-1:~/Downloads$ openssl rsa -in fake.keys -pubout -out fake.pub
writing RSA key
hibaghiba-VMware20-1:~/Downloads$ openssl pkeyutl -verify -in input -inkey fake.pub -pub
in -digest SHA-256 -rawin -sigfile input.sig
Signature Verification Failure
C06CD1495DFC0000:error:0200008A:rsa routines:RSA_padding_check_PKCS1_type_1:invalid padd
ing:../crypto/rsa/rsa_pk1.c:79:
C06CD1495DFC0000:error:02000072:rsa routines:rsa_ssl_public_decrypt:padding check fail
e:../crypto/rsa/rsa_ssl.c:796:
C06CD1495DFC0000:error:1C800004:Provider routines:rsa_verify_directly:RSA lib:../provide
rs/implementations/signature/rsa_sig.c:1041:

```

This happens because the signature was created with a different private key. Only the matching public key can verify it.

- Wrong Hash Algorithm

Verification failed when using a different hash algorithm. This is because the same algorithm must be used for both signing and verification

```
hiba@hiba-VirtualBox-1:~/Downloads$ openssl pkeyutl -verify -in input -inkey hiba-sig.pub  
-pubin -digest SHA-1 -rawin -sigfile input.sig  
Signature Verification Failure  
C04C250925FE0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa  
/rsa_sign.c:442:  
C04C250925FE0000:error:1C800004:Provider routines:rsa_verify_directly:RSA lib:../provide  
rs/implementations/signature/rsa_sig.c:1041:
```

This happens because the same hash algorithm must be used for both signing and verification.

- Signature File Test:

When using a modified or different signature file, verification fails because the signature no longer matches the message and public key.

This happens because the signature file must match the original message. If it is changed, the verification fails.

Did everything work as expected?

Yes, everything worked as expected. The correct file and key verified successfully, while modified input, wrong key, wrong hash algorithm, and incorrect signature all caused verification failure.

- Small File Test

A small file was signed and successfully recovered using message recovery. The recovered file matched the original file.

```

hiba@hiba-VMware20-1:~/Downloads$ echo "Small message for recovery" > input2
hiba@hiba-VMware20-1:~/Downloads$ ls -l input2
-rw-rw-r-- 1 hiba hiba 27 May  4 17:18 input2
hiba@hiba-VMware20-1:~/Downloads$ openssl pkeyutl -sign -in input2 -out input2.sig -inkey hiba-sig.keys
hiba@hiba-VMware20-1:~/Downloads$ openssl pkeyutl -verifyrecover -inkey hiba-sig.pub -pubin -in input2.sig -out input2.rec
hiba@hiba-VMware20-1:~/Downloads$ diff input2 input2.rec
hiba@hiba-VMware20-1:~/Downloads$

```

- Large File Test

Signing a large file failed because RSA has a size limit. In message recovery, the system works directly with the message instead of a hash. If the message is too big, it cannot be processed.

```

hiba@hiba-VMware20-1:~/Downloads$ for i in {1..200}; do echo "Big message test" >> bigfile; done
hiba@hiba-VMware20-1:~/Downloads$ ls -l bigfile
-rw-rw-r-- 1 hiba hiba 3400 May  4 17:21 bigfile
hiba@hiba-VMware20-1:~/Downloads$ openssl pkeyutl -sign -in bigfile -out bigfile.sig -inkey hiba-sig.keys
Error: The non-raw input data length 3400 is too long - max supported hashed size is 64
hiba@hiba-VMware20-1:~/Downloads$

```

- Different Public Key Test:

When using a different public key to verify or recover the message, the process failed. This is because the signature was created using a specific private key and can only be verified with the matching public key.

This happens because only the correct public key can verify or recover the message.

2 – Public Key Infrastructure (PKI)

- CA Certificate Creation

PKI is important because it helps people trust each other online. Without it, anyone could pretend to be someone else. The Certificate Authority makes sure that a public key belongs to the correct person or website.

A self-signed Certificate Authority (CA) certificate was created. This certificate is used to sign other certificates and establish trust.

```
hibakiba@hibakiba:~$ cd ~/Downloads$ openssl req -x509 -out hibaca.key
hibakiba@hibakiba:~$ cd ~/Downloads$ openssl req -x509 -out hibaca.cer
t -days 3650
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:no
State or Province Name (full name) [Some-State]:Agder
Locality Name (eg, city) []:Kristiansand
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Noroff
Organizational Unit Name (eg, section) []:NUC
Common Name (e.g. server FQDN or YOUR name) []:shiba CA
Email Address []:shibadark12085@gmail.com
hibakiba@hibakiba:~$ cd ~/Downloads$ openssl x509 -in hibaca.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            33:25:4b:72:cb:04:cd:44:e7:d2:4b:face:4c:9d:40:5f:0e:00:43
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=no, ST=Agder, L=Kristiansand, O=Noroff, OU=NUC, CN=shiba CA, emailAddr=
sshshibadark12085@gmail.com
        Validity
            Not Before: May  4 15:35:37 2026 GMT
            Not After : May  1 15:35:37 2036 GMT
        Subject: C=no, ST=Agder, L=Kristiansand, O=Noroff, OU=NUC, CN=shiba CA, emailAddr=
sshshibadark12085@gmail.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (4096 bit)
            Modulus:
                00:cd:ab:64:44:d1:93:0f:73:12:24:ac:dc:0f:ce:
                19:4f:5d:0e:3f:bd:46:0d:95:ad:33:0e:3a:b0:0c:
                23:f0:f0:31:60:0e:05:c9:1b:b0:72:4e:8f:d8:b1:
                e7:c7:56:07:f0:79:37:be:c3:d2:3a:44:b0:ce:39:
                3c:21:a5:25:12:af:04:04:97:00:4f:01:90:43:e5:
                36:b6:cf:0e:04:04:15:0c:26:a8:54:d8:75:4c:cf:
                ee:b0:c6:b6:80:b7:26:ef:f8:22:ca:3a:da:73:45:
                e8:b0:c6:b6:80:b7:26:ef:f8:22:ca:3a:da:73:45:
                b9:0f:f3:c9:5d:27:23:c2:f0:37:4b:3c:db:89:58:
                77:f7:68:04:24:f1:75:e5:15:1a:64:77:c7:8a:2d:
                e4:da:13:c1:09:19:15:41:13:97:d0:00:40:aec:2:00:
                d3:1f:f9:58:94:39:17:0e:58:2e:45:3f:27:09:fe:
                95:ce:fd:ad:df:07:92:ed:99:dd:84:f0:9e:28:1f:
                2f:56:9b:09:a5:a4:01:0e:06:be:b6:7d:22:40:ff:
                5a:1b:99:70:fe:33:25:50:01:06:03:0e:0f:4d:f3:
                7c:09:55:14:49:f3:6a:be:7b:5f:9b:e8:15:ab:e1:
                3d:32:7a:dc:d7:a0:00:59:79:d5:af:a7:7b:3b:54:
                c6:71:27
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                14:46:F2:A7:3E:22:22:A5:FA:95:66:09:C4:C2:27:EE:D1:07:1C:4A
            X509v3 Authority Key Identifier:
                14:46:F2:A7:3E:22:22:A5:FA:95:66:09:C4:C2:27:EE:D1:07:1C:4A
            X509v3 Basic Constraints: critical
                CA:TRUE
        Signature Algorithm: sha256WithRSAEncryption
        Signature Value:
            73:57:2f:b0:d8:58:29:1b:42:5e:56:d9:df:4a:e6:f0:99:b3:
            0d:24:0c:01:77:17:be:ad:f9:60:05:c0:90:f0:57:b7:40:a7:
            03:7c:07:01:65:0a:ca:b0:0a:ea:33:b4:13:a4:ef:a0:5e:0a:
            b0:f0:50:0e:5f:06:d0:9c:b4:cf:45:2e:8f:e4:8c:13:70:b4:
            0e:1a:ae:66:76:b2:2b:39:e9:3c:0f:b5:90:d7:a9:9c:75:f0:
            e2:19:a3:07:2d:c4:29:a1:0e:32:2a:c3:15:04:be:77:e5:fa:
            6c:17:2d:0a:0a:ec:3d:ab:0e:b1:06:1b:b2:b5:9d:72:ac:f7:
            0b:8a:e2:4f:71:cb:f8:5d:04:07:68:ea:1b:23:dc:3f:b1:90:
            4d:0d:1c:0c:e0:59:c1:d5:72:b5:e0:9a:59:d0:b1:bc:07:fa:
            0c:47:08:cb:31:dc:0a:00:a1:59:03:0e:09:2e:53:1d:2e:97:
            be:21:b6:20:e2:0c:9a:0d:f3:7f:5c:9f:b2:ff:71:90:5f:df:
            02:18:c0:47:68:40:10:0b:41:fe:50:67:00:6c:01:ab:ea:08:
            b0:64:94:50:1f:40:22:0d:fb:a7:0d:ef:a4:9e:cb:a3:b1:e0:
            d3:12:2a:f0:07:01:1a:7a:56:91:09:9d:e5:ca:ae:7b:04:06:
            b3:c6:6a:0f:93:db:10:d1:14:cf:b1:dd:aed:04:ea:45:5c:4e:
            99:5b:f3:59:9c:dc:ac:72:a2:04:b4:b4:2a:51:e7:a6:b2:1a:c2:
            cf:f5:20:cc:a7:b1:fc:c7:25:7f:0e:cc:75:d2:41:04:ad:df:
            97:a7:7a:7a:0b:07:c4:1f:c2:46:00:7a:7c:06:31:cd:09:
            7c:1e:0e:56:df:e9:50:fbed:5b:93:3d:3d:06:0c:cc:0e:1b:
            54:9a:3f:4f:12:39:5f:bd:2d:eb:5b:e7:26:42:7b:0e:0c:ca:
            0e:50:00:43:10:a4:1d:07:0e:0a:2e:e5:aa:62:21:a9:19:49:
            a1:14:70:40:04:f7:0d:05:4d:c2:be:2b:1e:45:07:a0:33:57:
            e0:81:7b:b7:3b:24:0d:1f:bd:7c:0f:97:31:03:59:00:52:b7:
            b7:75:0c:f6:08:e3:78:cf:bd:83:57:f5:b3:2d:28:64:35:5c:
            7f:27:09:bd:c8:d7:b0:4a:d2:07:60:09:ee:47:2a:0d:03:10:
            f0:ac:b6:04:19:09:71:06:1e:cb:b7:0e:70:d3:aa:5a:ef:f2:
            c7:a3:10:a1:9c:1a:1c:77:43:bc:f3:40:65:da:8c:f1:01:0e:
            73:fb:28:0f:50:b3:39:02:09:a9:fd:40:45:4b:2d:35:76:64:
            3d:07:b0a20:ef:1a:94:7c
```

- Certificate Inspection

The certificate contains important fields such as:

- Subject (owner)
- Issuer (CA)
- Validity period
- Public key
- Signature algorithm

The public exponent was 65537.

- CSR (Certificate Signing Request)

A CSR was created containing identity information and a public key. This request must be signed by a CA to become trusted.

- Comparison between CSR and Certificate:

A CSR contains identity information and a public key, but it is not trusted because it is not signed. A certificate contains similar information but is signed by a Certificate Authority, making it trusted and verifiable.

- Certificate Issuing

The CSR was signed by the CA to create a certificate. The message “database updated” confirms success.

```
hiba@hiba-VMware20-1:~/Downloads$ sudo nano /usr/lib/ssl/openssl.cnf
[sudo: authenticate] Password:
hiba@hiba-VMware20-1:~/Downloads$ ls /home/hiba/myCA
certs  crlnumber  index.txt  private
crl    hibaca.crt  newcerts  serial
hiba@hiba-VMware20-1:~/Downloads$ ls /home/hiba/myCA/private
hibaca.keys
hiba@hiba-VMware20-1:~/Downloads$
```

- Certificate Verification

The certificate was verified using the CA certificate, and the result was successful.

```
hiba@hiba-VMware20-1:~/Downloads$ openssl verify -CAfile ~/myCA/hibaca.crt hiba.crt
hiba.crt: OK
```

This shows that the certificate is trusted

because it was signed by a valid Certificate Authority.

- Final Certificate Details

The certificate shows:

- Subject = user
- Issuer = CA
- CA flag = FALSE (not a CA certificate)

```
h1haghi@h1haghi:~/Downloads$ openssl x509 -in hba.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            64:14:76:fc:64:46:1f:49:1e:35:2d:92:ff:5a:03:cf:41:7b:46:ff
        Signature Algorithm: sha256withRSAEncryption
        Issuer: C=us, ST=ny, O=Lakeittanand, OU=Off, CN=hba CA, email=sshibadark1205@gmail.com
        Validity
            Not Before: May  4 16:16:03 2020 GMT
            Not After : May  4 16:16:03 2027 GMT
        Subject: C=us, ST=ny, OU=Off, CN=hba CA, email=sshibadark1205@gmail.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (3072 bit)
            Modulus:
                00:c0:4c:1b:95:fb:02:04:02:10:e0:d1:b4:10:13:
                a7:22:07:79:05:a6:9e:43:1d:0a:ba:fe:ff:0c:5d:
                85:fe:03:6c:af:83:6d:0b:0b:57:07:10:a0:07:30:
                dc:1e:02:04:0b:14:0e:4b:17:21:5b:40:6d:37:8b:
                90:9c:0a:16:05:13:03:d1:1a:c3:0e:00:71:40:20:
                40:20:0b:01:10:45:e7:b2:3f:0e:3c:93:d1:f6:03:
                07:92:10:f6:c0:aa:14:1d:27:70:62:07:d2:14:0a:
                20:e4:52:17:0a:1f:10:22:04:0e:10:07:ec:ed:05:
                40:2e:5d:52:cc:47:0f:a7:6c:95:ec:b3:09:e1:b4:
                17:d1:0e:f3:20:35:9f:6d:1c:02:0a:0a:04:f0:02:
                51:32:1d:ac:2c:13:52:96:2c:12:0a:06:7f:0d:24:
                0d:a7:b3:06:11:1f:03:0b:00:15:55:05:20:f6:0e:
                40:b7:56:c3:06:0c:52:3a:d1:56:3c:0e:2d:7e:90:
                5c:22:a9:05:a7:50:b4:0d:ab:93:f5:ec:c7:c2:0b:
                00:7e:0f:f6:02:0a:9e:c3:4d:09:01:ec:16:4e:0d:
                c3:e9:0b:15:0a:f2:43:27:f1:1f:07:70:03:0b:00:
                06:1e:74:f0:c5:70:05:36:0b:2a:11:03:ed:c8:2a:
                3c:e9:0b:05:05:b3:03:1d:0f:0f:f1:b7:c0:00:
                02:7e:00:a3:07:0e:47:3f:33:26:4f:40:20:0a:00:
                70:0b:c0:70:00:00:0c:13:11:7f:90:05:40:33:
                e7:04:ad:05:27:70:70:02:00:07:30:03:f3:51:b3:
                fc:a7:0e:49:1a:0f:90:b0:05:0f:00:40:30:90:20:
                c5:4c:73:12:a4:2d:0e:00:0a:0b:07:00:07:7c:40:
                7f:30:1f:0d:15:ab:5f:74:91:1a:00:00:7f:90:70:
                40:ac:00:70:01:cc:05:04:03:0c:e4:30:a3:07:d3:
                c9:0f:14:05:2e:10:0a:0b:05:05:07
            Exponent: 65537 (0x10001)
h1haghi@h1haghi:~/Downloads$
```

- Manual Certificate Validation:

I exported certificates from the Noroff website and verified them using OpenSSL. This demonstrated how certificate chains work and how trust is established through intermediate and root Certificate Authorities.

This shows how trust works in real systems, where certificates are checked step by step using a chain of trust.

How should a CA certificate be distributed?

A CA certificate should be distributed through a secure and trusted channel. Downloading certificates from untrusted sources can allow attackers to impersonate trusted entities.

- Website Certificate:

I repeated the CSR process for a hypothetical website. In this case, the Common Name (CN) field was set to a domain name instead of a personal name. This is because website certificates identify domains rather than individuals.

3 – Keystream Reuse Attack

- Simulator Execution

The simulator was successfully executed.

```
hibaghiba-VMware20-1:~/Downloads$ cd ~/Downloads
hibaghiba-VMware20-1:~/Downloads$ ls
bigfile      cc          hiba-sig.pub  input      input2.enc  md5col.zip
bigfile.sig  'cc(1)'    hiba.crt     input.dec  input2.rec  msg1
blue.bmp     fake.keys  hiba.csr     input.enc  input2.sig  msg2
blue.cbc.gz  fake.pub   hiba.keys    input.sig  letter.ps   transfer.ps
blue.ecb.gz  hiba-sig.keys hiba.pub     input2     md5col
hibaghiba-VMware20-1:~/Downloads$ unzip cc.zip
unzip: cannot find or open cc.zip, cc.zip.zip or cc.zip.ZIP.
hibaghiba-VMware20-1:~/Downloads$ chmod +x cc
hibaghiba-VMware20-1:~/Downloads$ ./cc "Hiba"
qemu-x86_64: Could not open '/lib64/ld-linux-x86-64.so.2': No such file or directory
hibaghiba-VMware20-1:~/Downloads$ chmod +x cc
hibaghiba-VMware20-1:~/Downloads$ ./cc "Hiba"
qemu-x86_64: Could not open '/lib64/ld-linux-x86-64.so.2': No such file or directory
hibaghiba-VMware20-1:~/Downloads$
```

Issuing a Card

A new credit card was generated.

```
Weak credit card protection simulator
=====
By Nelson Uto, 2023

Issued card 2723 2766 8631 5164

Press any key to continue...
```

Database Dump

Encrypted credit card data was displayed.


```
Weak credit card protection simulator
=====
By Nelson Uto, 2023

Database:
=====
b62976cc0ba6a12c
9d5a95de338521ec
876db95ee5af013c
bb5cf14f4bf511fc
aa58832c71b2b2f3
e75af54e216eb730
cce661a73afa6f4
c62ba533e57f34e4
faebf27b3ae31326
4d1de6635e273d5
e84ac73cf5d775c1
```

- Attack Explanation

The attack works because the system uses the same key again and again. This is a big mistake. When the attacker knows both the plaintext and the encrypted version, they can use XOR to find the key. After finding the key, all other data can be decrypted.

- Recovered Values:

The plaintext card was 2723276686315164. The encrypted version was identified as the last entry in the database: e84ac73cf5d775c1. By XORing these values, the keystream was recovered: cf69e05a73e624a5.

- Result

After recovering the key, the system was successfully broken and all data was decrypted.

```
Weak credit card protection simulator
=====
By Nelson Uto, 2023

Success!! Congratulations!

Database
=====
7940 9696 7840 8589
5233 7584 4063 0549
4804 5904 9649 2599
7435 1115 3813 3559
6531 6376 0254 9656
2833 1514 5288 9395
0386 8640 0049 8251
0942 4569 9699 1041
3582 1221 4905 3783
8274 0639 2404 5770
2723 2766 8631 5164
```

This shows that even strong encryption can fail if it is used in the wrong way. In real life, this kind of mistake can cause serious security problems.

- This lab demonstrated:
- How digital signatures ensure integrity and authenticity
- How PKI creates trust through certificates
- How weak encryption (keystream reuse) can be exploited

This shows the importance of using secure cryptographic practices in real-world systems.